

SALESTORM — API Specification

1. Purpose

The SALESTORM API layer provides controlled access to product discovery, cart management, inventory reservation, checkout, payment, order processing, and shipment tracking.

The API design follows these principles:

- RESTful resource-oriented APIs
- Authentication and authorization
- Idempotency for critical operations
- Input validation
- Consistent HTTP status codes
- Secure communication through HTTPS
- Synchronous APIs for critical customer decisions
- Asynchronous events for downstream processing
- Correlation IDs for distributed tracing

2. Base API Configuration

Property	Specification
Protocol	HTTPS
API Style	REST
Format	JSON
Base URL	/api/v1
Authentication	JWT / OAuth 2.0
Authorization	Role-based access control
Content-Type	application/json
Idempotency	Required for reservation, payment and checkout
Tracing	X-Correlation-ID
Versioning	URI versioning
Common	Headers
Authorization	Bearer <access_token>
Content-Type	application/json
X-Correlation-ID	<unique-request-id>
Idempotency-Key	<unique-operation-key>

Idempotency-Key is mandatory for operations where repeating the same request could create a duplicate business transaction.

3. API Overview

Method	Endpoint	Purpose	Type
GET	/products	Product discovery	Synchronous
GET	/products/{productId}	Product details	Synchronous
GET	/products/{productId}/inventory	Inventory availability	Synchronous
POST	/carts	Create cart	Synchronous
POST	/carts/{cartId}/items	Add item to cart	Synchronous
POST	/reservations	Reserve inventory	Synchronous
GET	/reservations/{reservationId}	Reservation status	Synchronous
DELETE	/reservations/{reservationId}	Release reservation	Synchronous
POST	/checkout	Start checkout	Synchronous
POST	/payments	Initiate payment	Synchronous
GET	/payments/{paymentId}	Payment status	Synchronous
POST	/orders	Create/confirm order	Synchronous/Event-driven
GET	/orders/{orderId}	Order details	Synchronous
POST	/orders/{orderId}/cancel	Cancel order	Synchronous
GET	/shipments/{shipmentId}	Shipment tracking	Synchronous
POST	/webhooks/payment	Payment gateway callback	Event-driven

4. Product APIs

Get Products

Request:

GET /api/v1/products

Query Parameters:

- category
- search
- page
- limit
- sort

Response:

```
{  
  "data": [  
    {  
      "product_id": "PROD-001",  
      "name": "Product X",  
      "price": 4999.00,  
      "currency": "INR",  
      "available": true  
    }  
  ],  
  "page": 1,  
  "limit": 20,  
  "total": 1  
}
```

Status Codes

200 OK

400 Bad Request

401 Unauthorized

5. Inventory APIs

Inventory is the most critical consistency boundary in SALESTORM.

Check Inventory

GET /api/v1/products/{productId}/inventory

Response

```
{  
  "product_id": "PROD-001",  
  "available_quantity": 100,  
  "reserved_quantity": 0,  
  "sold_quantity": 0  
}
```

6. Create Inventory Reservation

Request:

POST /api/v1/reservations

Headers:

Authorization: Bearer <token>

Idempotency-Key: RESERVE-USER123-PROD001

X-Correlation-ID: CORR-10001

Request Body

```
{  
  "product_id": "PROD-001",  
  "quantity": 1,  
  "customer_id": "CUS-001"  
}
```

Successful Response:

```
{  
  "reservation_id": "RES-10001",  
  "product_id": "PROD-001",  
  "quantity": 1,  
  "status": "RESERVED",  
  "expires_at": "2026-10-05T10:30:00Z"  
}
```

Out-of-Stock Response:

```
{  
  "error": {  
    "code": "OUT_OF_STOCK",  
    "message": "Requested quantity is not available"  
  }  
}
```

Status Codes:

201 Created Reservation successfully created

400 Bad Request Invalid request

401 Unauthorized Authentication failed

409 Conflict Stock unavailable / duplicate conflict

429 Too Many Requests Rate limit exceeded

500 Internal Server Error Unexpected failure

7. Idempotent Reservation

If the same request is sent again using the same Idempotency-Key, the system does not create another reservation.

Response:

```
{  
  "reservation_id": "RES-10001",  
  "status": "RESERVED",  
  "is_duplicate": true  
}
```

Key Principle:

Same Idempotency-Key

↓

Existing transaction found

↓

Return existing result

↓

NO duplicate reservation

8. Release Reservation

DELETE /api/v1/reservations/{reservationId}

Response

```
{  
  "reservation_id": "RES-10001",  
  "status": "RELEASED",  
  "released_quantity": 1  
}
```

Used When

Payment fails

Reservation expires

Customer cancels checkout

System compensation is required

9. Checkout API

Checkout acts as the orchestration entry point for the purchase workflow.

POST /api/v1/checkout

Headers:

Authorization: Bearer <token>

Idempotency-Key: CHECKOUT-USER123-ORDER001

X-Correlation-ID: CORR-20001

Request:

```
{  
  "customer_id": "CUS-001",  
  "items": [  
    {  
      "product_id": "PROD-001",  
      "quantity": 1  
    }  
  ],  
  "payment_method": "UPI"  
}
```

Response:

```
{  
  "checkout_id": "CHK-10001",  
  "reservation_id": "RES-10001",  
  "payment_id": "PAY-10001",  
}
```

```
"status": "PAYMENT_PENDING"
}
```

10. Payment API

Initiate Payment:

POST /api/v1/payments

Request

```
{
  "order_reference": "CHK-10001",
  "amount": 4999.00,
  "currency": "INR",
  "payment_method": "UPI"
}
```

Response

```
{
  "payment_id": "PAY-10001",
  "status": "PAYMENT_PENDING",
  "amount": 4999.00
}
```

11. Payment Status

GET /api/v1/payments/{paymentId}

Successful Response

```
{
  "payment_id": "PAY-10001",
  "status": "SUCCESS",
}
```

```
"transaction_reference": "TXN-938274",  
"amount": 4999.00  
}
```

Possible Payment States:

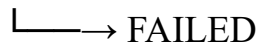
INITIATED



PENDING



SUCCESS



FAILED

For timeout:

PENDING



TIMEOUT



RECONCILIATION

12. Payment Idempotency

Every payment request must contain an idempotency key.

POST /api/v1/payments

Idempotency-Key: PAY-USER123-CHECKOUT1001

If the same request is repeated:

Existing Payment Found



Return Existing Payment



Do NOT charge customer again

This protects against:

- Double-clicks
- Network retries
- Client retries
- Gateway timeout retries
- Duplicate API requests

13. Payment Gateway Webhook

External payment providers notify SALESTORM through a webhook.

POST /api/v1/webhooks/payment

Example:

```
{  
  "event_id": "EVT-10001",  
  "payment_id": "PAY-10001",  
  "status": "SUCCESS",  
  "transaction_reference": "TXN-938274"  
}
```

The event is validated and processed idempotently.

14. Order API

Create Order:

POST /api/v1/orders

Request

```
{  
  "reservation_id": "RES-10001",  
  "payment_id": "PAY-10001",  
  "customer_id": "CUS-001"  
}
```

Response

```
{  
  "order_id": "ORD-10001",  
  "status": "CONFIRMED",  
  "payment_status": "SUCCESS",  
  "reservation_status": "CONFIRMED"  
}
```

15. Order Status API

GET /api/v1/orders/{orderId}

Response

```
{  
  "order_id": "ORD-10001",  
  "status": "PROCESSING",  
  "payment_status": "SUCCESS",  
  "items": [  
    {  

```

```
    "product_id": "PROD-001",  
    "quantity": 1  
  }  
]  
}
```

Order Lifecycle:

CREATED



PAYMENT_PENDING



CONFIRMED



PROCESSING



SHIPPED



OUT_FOR_DELIVERY



DELIVERED

16. Cancel Order

POST /api/v1/orders/{orderId}/cancel

Request

```
{  
  "reason": "CUSTOMER_REQUEST"  
}
```

Response

```
{  
  "order_id": "ORD-10001",  
  "status": "CANCELLED",  
  "refund_status": "INITIATED"  
}
```

17. Shipment API

Get Shipment Tracking

GET /api/v1/shipments/{shipmentId}

Response

```
{  
  "shipment_id": "SHIP-10001",  
  "order_id": "ORD-10001",  
  "status": "OUT_FOR_DELIVERY",  
  "estimated_delivery": "2026-10-07"  
}
```

18. Asynchronous Events

Not every operation should be handled synchronously.

SALESTORM uses asynchronous events for operations that don't need to block the customer request.

Core Events	Event Producer	Consumer
ReservationCreated	Inventory Service	Checkout
ReservationExpired	Reservation Service	Inventory
PaymentSucceeded	Payment Service	Order Service

PaymentFailed	Payment Service	Inventory/Order
OrderConfirmed	Order Service	Shipment/Notification
ShipmentCreated	Shipment Service	Notification
OrderDelivered	Shipment Service	Notification

19. Critical Event Flow

Payment Success + Order Service Failure

Payment Service



PaymentSucceeded



Transactional Outbox



Message Broker



Order Service



Order Created

If Order Service is unavailable:

Message Broker



Retry



Order Service Recovery



Idempotent Processing



Order Confirmed

This prevents successful payments from being lost.

20. Common Error Response

All APIs use a consistent error structure.

```
{  
  "error": {  
    "code": "OUT_OF_STOCK",  
    "message": "Product is no longer available",  
    "correlation_id": "CORR-10001",  
    "timestamp": "2026-10-05T10:25:30Z"  
  }  
}
```

Common Error Codes

HTTP	Status	Error Meaning
400	INVALID_REQUEST	Invalid input
401	UNAUTHORIZED	Authentication required
403	FORBIDDEN	Insufficient permission
404	NOT_FOUND	Resource doesn't exist
409	OUT_OF_STOCK	Inventory unavailable
409	DUPLICATE_REQUEST	Idempotency conflict
429	RATE_LIMITED	Too many requests
500	INTERNAL_ERROR	Internal failure
502	PAYMENT_PROVIDER_ERROR	External gateway failure

503 SERVICE_UNAVAILABLE Temporary service failure

21. API Security

All APIs are protected using:

- HTTPS/TLS
- JWT/OAuth authentication
- Role-based authorization
- Input validation
- API rate limiting
- Request size limits
- Idempotency keys
- Secure secrets management
- Audit logging
- Payment-tokenization instead of storing sensitive card information

22. API Reliability Strategy

Critical APIs use:

Timeout - Prevent requests from waiting indefinitely.

Retry - Used only for safe/transient failures.

Circuit Breaker - Protects the system from unhealthy external dependencies.

Idempotency - Prevents duplicate business operations.

Correlation ID - Allows a single transaction to be traced across services.

Dead-Letter Queue - Stores messages that repeatedly fail processing.

23. Synchronous vs Asynchronous API Design

Operation	Communication	Reason
Product search	Synchronous	User needs immediate result
Inventory reservation success/failure	Synchronous	User needs immediate

Payment initiation Synchronous Start transaction immediately

Payment confirmation Event-driven Gateway may respond asynchronously

Order creation Event-driven / synchronous boundary Reliable after payment

Notification Asynchronous Doesn't need to block purchase

Shipment update Asynchronous Independent downstream process

Analytics Asynchronous Not part of critical transaction

Design Principle

Keep the critical purchase decision synchronous; move non-critical downstream work to asynchronous processing.

24. Critical Purchase API Flow

Customer

|

| POST /checkout



API Gateway

|



Checkout Service

|



Inventory Service

|

| Atomic Reservation



Inventory Database

|



Payment Service

|



Payment Gateway

|



Transactional Outbox

|



Message Broker

|



Order Service

|



Confirmed Order

25. API Design for the 10,000 → 100 Scenario

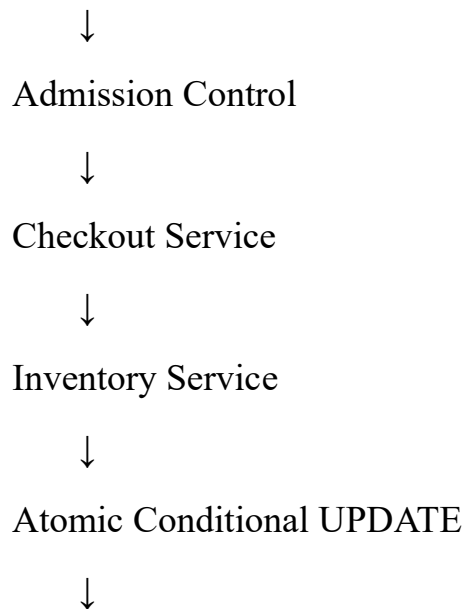
When 10,000 customers simultaneously call:

POST /api/v1/checkout the system follows:

10,000 Requests



API Gateway



100 Reservations
9,900 Rejected
Oversold = 0

The API layer therefore supports the core SALESTORM guarantee:

No matter how many concurrent purchase requests arrive, the API and inventory transaction design ensures that successful reservations cannot exceed the available inventory.

26. API Design Principles

The SALESTORM API architecture follows these principles:

- ✓ RESTful resource design
- ✓ Versioned APIs
- ✓ Authentication and authorization
- ✓ Idempotency for critical operations
- ✓ Atomic inventory operations
- ✓ Consistent error responses
- ✓ Synchronous processing for critical decisions
- ✓ Asynchronous event processing for downstream workflows

- ✓ Timeout, retry and circuit-breaker protection
- ✓ Correlation IDs for distributed tracing
- ✓ Input validation and rate limiting
- ✓ Secure payment integration